

Energy-Efficient Keyword Spotting for Autonomous Microbots using Gated CNN on Edge

Ayush

Dept. of Computer Science Engineering
IIIT Hyderabad
Hyderabad, India

Pranav

Dept. of Computer Science Engineering
IIIT Hyderabad
Hyderabad, India

Yashav

Dept. of Computer Science Engineering
IIIT Hyderabad
Hyderabad, India

Raza

Dept. of Computer Science Engineering
IIIT Hyderabad
Hyderabad, India

Abstract—Voice-controlled micro-robots typically rely on cloud-based processing, which introduces latency and privacy concerns. Edge-based solutions are constrained by limited battery life and computational resources. This letter presents an energy-efficient Keyword Spotting (KWS) system implemented on an ESP32-S3 microcontroller for a differential drive microbot. We propose a multi-stage gating mechanism combining Signal-to-Noise Ratio (SNR), Root Mean Square (RMS) energy, and Zero-Crossing Rate (ZCR) to filter non-speech audio before invoking a lightweight Convolutional Neural Network (CNN). The system achieves a command recognition latency of approximately 102 ms and extends battery life by preventing unnecessary model inference. Experimental results demonstrate robust offline control with high accuracy for four directional commands.

Index Terms—Keyword Spotting, Edge AI, Embedded Systems, Micro-robotics, Convolutional Neural Networks.

I. INTRODUCTION

Voice User Interfaces (VUI) have become a dominant modality for human-machine interaction, driven by the ubiquity of smart speakers and virtual assistants. At the core of these systems lies Keyword Spotting (KWS), a classification task designed to detect a predefined set of words or phrases within a continuous stream of audio. Unlike Large Vocabulary Continuous Speech Recognition (LVCSR) which transcribes full sentences, KWS focuses on identifying specific triggers, making it the critical first step in command-and-control applications.

Deploying robust KWS on the “edge”—specifically on microcontrollers (MCUs)—presents a distinct set of challenges compared to server-grade implementations. Edge devices, such as the ESP32-S3 used in this work, operate under severe resource constraints. They typically possess limited Static RAM (SRAM) for storing intermediate tensors and Flash memory for model weights. Furthermore, they lack the powerful GPUs found in cloud servers, relying instead on low-power CPUs or specialized vector instructions. Achieving high accuracy with low latency while fitting the model into these constraints requires careful architectural design and optimization, such as quantization.

A critical bottleneck in battery-operated edge KWS is energy consumption. Deep learning models, particularly Convolutional Neural Networks (CNNs), involve millions of multiply-accumulate (MAC) operations per inference. In a typical environment, a robot spends the vast majority of its time in silence or background noise. An “always-on” system that runs a full CNN inference on every audio frame is inherently inefficient, draining the battery rapidly even when no commands are spoken.

To address this, Our edge KWS system leverages a “gating” mechanism or Voice Activity Detection (VAD) stage. This technique introduces a hierarchical processing pipeline. A low-complexity, low-power algorithm continuously monitors the audio signal for speech-like characteristics. Only when this gate detects potential speech does it wake the downstream neural network to perform the expensive classification. This cascaded approach offers dual benefits: it drastically reduces the average power consumption by keeping the MCU in a lower-power state during silence, and it reduces false positives by filtering out non-speech acoustic events before they reach the classifier.

In this letter, we present a complete hardware and software implementation of an offline, voice-controlled microbot. We focus on optimizing the trade-off between recognition accuracy and energy consumption through a robust gating strategy and a quantized neural network.

This work makes the following contributions:

- 1) A lightweight CNN architecture optimized for the ESP32-S3 to recognize four navigational commands (Forward, Backward, Left, Right).
- 2) A robust multi-metric gating algorithm using Signal-to-Noise Ratio (SNR), Root Mean Square (RMS) energy, and Zero-Crossing Rate (ZCR) to filter noise and silence, significantly reducing computational load.
- 3) A complete offline implementation demonstrating real-time control with <120 ms latency and safety features.

II. SYSTEM ARCHITECTURE

The system is built around the Espressif ESP32-S3 SoC, which features a dual-core Xtensa LX7 processor with vector instructions (AI acceleration) and BLE 5.0.

A. Hardware Setup

The robot employs a differential drive configuration with N20 motors and a DRV8833 driver. Audio is captured via an INMP441 MEMS microphone using the I2S interface. An HC-SR04 ultrasonic sensor provides obstacle avoidance safety. Power is supplied by 4xAA batteries, necessitating strict energy management.

B. Software Pipeline

The processing pipeline consists of five stages:

- 1) **Audio Acquisition:** Continuous buffering of 16kHz audio samples.
- 2) **Gating (VAD):** Fast rejection of non-speech frames.
- 3) **Feature Extraction:** Conversion of raw audio to spectrograms.
- 4) **Inference:** 8-bit quantized CNN execution.
- 5) **Post-Processing:** Confidence thresholding and command execution.

III. PROPOSED METHOD

A. Multi-Stage Gating Mechanism

To minimize power consumption, we implement a “Speech Gate” that analyzes the raw audio buffer before feature extraction. The gate computes three metrics on the raw PCM data: Root Mean Square (RMS) energy, Zero-Crossing Rate (ZCR), and Signal-to-Noise Ratio (SNR).

The system maintains a running estimate of the background noise level (N_{est}) and noise ZCR (Z_{est}), updated during non-speech frames using a first-order low-pass filter with an adaptive coefficient α :

$$N_{est}[t] = (1 - \alpha) \cdot N_{est}[t - 1] + \alpha \cdot \text{AvgAbs}[t] \quad (1)$$

where α is set to 0.05 for noise-like frames and 0.005 otherwise, ensuring the noise floor adapts quickly to environmental changes but remains stable during speech.

The gate transitions to an “open” state only if a set of strict conditions are met for $k_{min} = 2$ consecutive frames:

$$(SNR > T_{SNR} \vee RMS > T_{RMS}) \wedge (ZCR < T_{ZCR}) \quad (2)$$

where $T_{SNR} = 1.25$, $T_{RMS} = 58.5$, and $T_{ZCR} = 0.28$. A secondary “Strong” threshold set ($T_{SNR_High} = 2.15$, $T_{RMS_High} = 106.5$) allows for faster triggering on clear, loud commands. Once triggered, the gate remains open for a “hangover” period of 4 frames to capture the tail of the utterance.

B. Feature Extraction

Upon gate activation, the system processes a 1-second audio buffer ($L = 16000$ samples). We compute a spectrogram using a sliding Hamming window of size $W = 320$ (20 ms) with a step size of $S = 160$ (10 ms). For each window, a 512-point Fast Fourier Transform (FFT) is computed.

To reduce dimensionality, the magnitude squared of the FFT bins is pooled using average pooling with a kernel size of 6. The result is log-transformed to compress the dynamic range:

$$S[t, f] = \log_{10} \left(\frac{1}{P} \sum_{i=0}^{P-1} |X[t, f \cdot P + i]|^2 + \epsilon \right) \quad (3)$$

This yields a compact input tensor of shape (99, 43, 1), representing time, frequency, and channels respectively.

C. Lightweight CNN Architecture

We designed a compact CNN to fit within the ESP32-S3’s 384KB SRAM. The model takes a $99 \times 43 \times 1$ spectrogram input and processes it through the following layers:

- 1) **Conv2D:** 4 filters, 3×3 kernel, ReLU activation, L2 regularization (0.001).
- 2) **MaxPool2D:** 2×2 pooling.
- 3) **Conv2D:** 4 filters, 3×3 kernel, ReLU activation, L2 regularization (0.001).
- 4) **MaxPool2D:** 2×2 pooling.
- 5) **Flatten** followed by **Dropout (0.1)**.
- 6) **Dense:** 80 units, ReLU activation, L2 regularization (0.001), Dropout (0.1).
- 7) **Output Dense:** 5 units (Forward, Backward, Left, Right, Unknown), Softmax activation.

The model contains a total of **77,473 parameters**. It is trained using the Adam optimizer and Categorical Cross-Entropy loss. Post-training, the model is converted to TensorFlow Lite for Microcontrollers (TFLite Micro) with full integer quantization (int8). This reduces the model size to approximately 81 kB, allowing it to reside entirely in the MCU’s internal flash and execute efficiently in RAM.

D. Post-Processing & Safety

Raw model probabilities are filtered to prevent false positives. A command C_i is accepted only if its probability $P(C_i)$ exceeds a class-specific threshold T_i (e.g., $T_{fwd} = 0.70$, $T_{left} = 0.50$) and the margin against the second-best class exceeds $M = 0.15$.

$$P(C_i) > T_i \quad \wedge \quad (P(C_i) - \max_{j \neq i} P(C_j)) > M \quad (4)$$

A cooldown period of 1000 ms is enforced after a successful detection to prevent repeated triggering from echoes.

IV. EXPERIMENTAL RESULTS

A. Model Evaluation

The model was trained on a custom dataset of the four target commands plus a set of “unknown” words and background noise. The training process achieved a validation accuracy of **92.87%**.

The confusion matrix (Table I) illustrates the model’s performance across all classes. The high diagonal values indicate robust classification, with minimal confusion between directional commands.

TABLE I
CONFUSION MATRIX (NORMALIZED)

True/Pred	Fwd	Back	Left	Right	Unk
Fwd	0.94	0.01	0.02	0.01	0.02
Back	0.01	0.93	0.01	0.02	0.03
Left	0.02	0.01	0.91	0.01	0.05
Right	0.01	0.02	0.01	0.92	0.04
Unk	0.03	0.02	0.03	0.02	0.90

B. Latency and Performance

The end-to-end latency from the end of a spoken command to motor actuation was measured at approximately **102 ms**. This falls well within the target of <120 ms for perceived real-time response.

C. Gating Efficiency

The gating mechanism successfully filters out background noise and silence. In a quiet environment, the gate remains closed >90% of the time, preventing the CPU from running the computationally intensive FFT and CNN inference. This significantly reduces the average current consumption.

D. Battery Life

On 4xAA batteries (approx. 2500mAh capacity), the robot achieves a runtime of:

- **Idle (Listening):** ~3.5 hours
- **Active (Moving):** ~2.25 hours (at 100% motor duty cycle)

E. Resource Usage

- **Model Size:** ~81 kB
- **Tensor Arena:** 50 kB
- **Inference Time:** <100 ms

V. CONCLUSION

We presented a fully offline, voice-controlled microbot system. By combining a low-cost gating stage with a quantized CNN, we achieved robust command recognition on a resource-constrained ESP32-S3. The system balances latency, accuracy, and power efficiency, making it suitable for low-power edge robotics applications. Future work will explore on-device learning to adapt to specific user voices.

REFERENCES

- [1] Y. Zhang et al., “Hello Edge: Keyword Spotting on Microcontrollers,” *arXiv preprint arXiv:1711.07128*, 2017.
- [2] P. Warden, “Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition,” *arXiv preprint arXiv:1804.03209*, 2018.